

Group Meow
Machine Learning BSMALÉA1KU
Fall Semester 2024

Machine Learning Exam Assignment Project

Group Members:

Matúš Stanko, matus@itu.dk
Lucie Navrátilová, luna@itu.dk
David Poda, dpod@itu.dk

IT UNIVERSITY OF CPH

IT University of Copenhagen
January 6th, 2025

1 Introduction

The purpose of this project is to explore different classification methods for determining the type of clothing from an image.

2 Dataset

The dataset contains 15,000 images of clothing items categorized into the following classes: T-shirt/top, Trouser, Pullover, Dress, Shirt. Out of the total, 10,000 images will be used to train the classification model, and 5,000 will be used for testing. These categories are equally represented in the dataset, each being around a 1/5 of the dataset.

The data is separated as follows, to separate features and labels:

```
X_train, y_train = train_data[:, :-1], train_data[:, -1]
X_test, y_test = test_data[:, :-1], test_data[:, -1]
```

3 Visualization

Each image is a 28×28 grayscale representation, with pixel values ranging from 0 (black) to 255 (white). The dataset structure includes 784 columns representing the pixel values, and the 785th column contains the label corresponding to the clothing category.

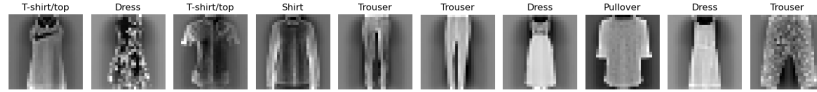


Figure 1: 10 randomly selected samples from the dataset.

3.1 Principal Component Analysis

To understand and visualize the dataset, we started by applying Principal Component Analysis (PCA). We visualized the variance explained by each principal component and plan to evaluate our classification models using 112 principal components, which explain 90% of variance and reduce the dimensionality by a significant amount.

We used `scikit-learn`'s PCA and applied it to our data, either the train or test data as needed:

```
pca = PCA(n_components = 112)
X_pca = pca.fit_transform(X)
```

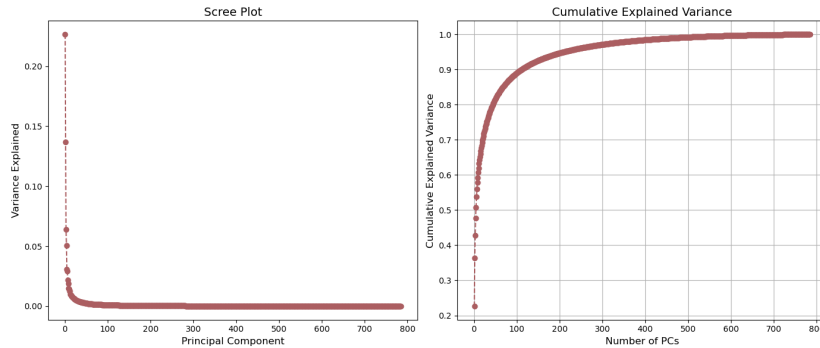


Figure 2: Scree plot and cumulative variance explained by a given number of principal components.

Using PCA also helped us visualize the dataset by reducing its dimensions. We created two plots: a 2D visualization with two principal components and a 3D visualization with three. These plots show a clear separation between trousers and dresses, while t-shirts/tops, pullovers and shirts are more mixed together. This overlap could make distinguishing between these three categories difficult for the classification.

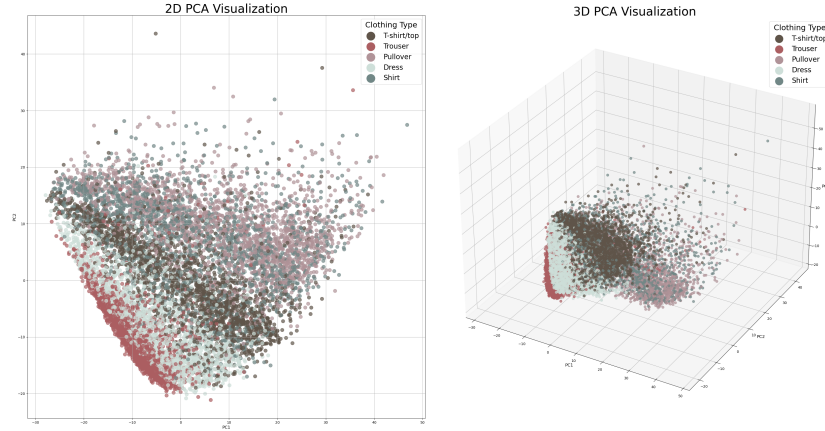


Figure 3: Visualizing the dataset using 2 and 3 principal components.

Using 112 principal components, we reconstructed several images to demonstrate how well this reduced dimensionality preserves the original data’s variance. The use of 112 components ensures that a significant portion of the variance is retained and keeps the most critical features of the images. As a result, the reconstructed images maintain a lot of similarity to the originals, validating the choice of 112 components as a balance between reducing complexity and still preserving enough details.



Figure 4: Original data and data reconstruction with 112 principal components

4 Classification methods

4.1 M1: Decision Trees

4.1.1 Introduction

Decision Trees were selected as the first machine learning model (M1) to classify the Fashion dataset. Both a custom implementation from scratch and a library-based implementation (scikit-learn) were used to explore the performance of the model. Additionally, experiments were conducted to compare the effects of using normalized and non-normalized pixel values (0-255 vs. 0-1).

4.1.2 Hyperparameter Tuning

1. **Max depth** Max depth controls the maximum number of levels in the decision tree. It plays a crucial role in balancing underfitting and overfitting. The graphs below show the effect of varying max depth and additional tuning.

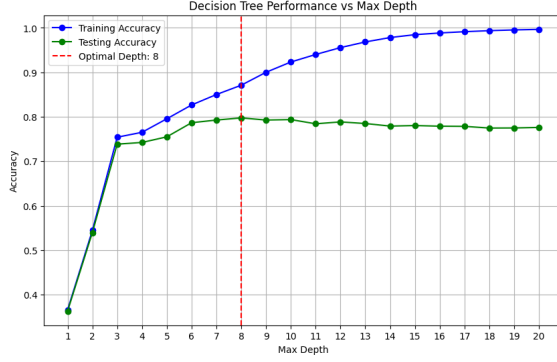


Figure 5: Training and testing accuracy for different max depth values.

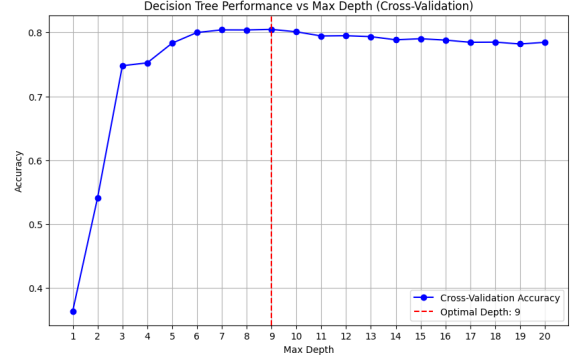


Figure 6: Accuracy for different max depth values using 5-fold crossvalidation.

Figure 5 shows that a max depth of 8 provides the best balance between training and testing accuracy, preventing overfitting. However, when cross-validation is used (Figure 6), the optimal max depth is identified as 9, suggesting a better generalization to unseen data. Based on these results, a max depth of 9 is chosen for further experiments as it is more robust across different splits of the data.

2. **Min samples leaf and Min samples split** The hyperparameters *min samples leaf* and *min samples split* control the minimum number of samples required for a split or a leaf node. These parameters influence the tree's complexity and help prevent overfitting. The graphs below show the effect of tuning *min samples leaf* and *min samples split* for different *max depth* values.

Max Depth	Min Split	Min Leaf	Accuracy
5	10	10	0.7566
9	100	10	0.8002
15	100	20	0.7984
20	100	20	0.7984
35	100	20	0.7984
50	100	20	0.7984

Table 1: Min Split and Min Leaf Combinations for Different Max Depth Values and Best Accuracies

Based on these results, the optimal combination of hyperparameters for further experiments is:

Hyperparameter	Value
Max Depth	9
Min Samples Split	100
Min Samples Leaf	10

Table 2: Optimal hyperparameter values for further experiments.

3. **Comparison of Gini and Entropy** Using the hyperparameters listed in Table 2, we evaluated the performance of the Decision Tree with two impurity functions: *Gini* and *Entropy*. The results are presented in the table below:

Based on the results, the *Gini* criterion achieved a higher accuracy (0.8002) compared to *Entropy* (0.7802). Therefore, the *Gini* criterion will be used for further experiments due to its better performance.

Criterion	Accuracy
Gini	0.8002
Entropy	0.7802

Table 3: Comparison of accuracy for Gini and Entropy impurity functions using the hyperparameters from Table 2.

Hyperparameter Observations Based on the hyperparameter tuning results, the following observations can be summarized:

Parameter	Observation
Max Depth	Optimal value is 9 (Figure 6), balancing generalization and complexity with an accuracy of 0.8002 , avoiding overfitting seen in deeper trees.
Min Samples Split	Best results with 100 (Table 2), ensuring sufficient samples before splitting to prevent overfitting.
Min Samples Leaf	10 (Table 2) improves reliability of leaf predictions and reduces overfitting.
Impurity Function	<i>Gini</i> outperforms <i>Entropy</i> with an accuracy of 0.8002 vs. 0.7802 (Table 3), making it the better choice for this dataset.

Table 4: Summary of hyperparameter tuning observations.

4.1.3 Implementations

Library Implementation To build a decision tree classifier using Python’s `scikit-learn` library, the process involves the following steps:

1. **Import Libraries and Load Data:** The required libraries are imported, and the dataset is loaded and prepared.
2. **Train the Decision Tree:** A `DecisionTreeClassifier` Class is initialized with optimal hyperparameters and trained.
3. **Evaluate the Classifier:** Predictions are made using the test set, and accuracy is calculated.

Custom Implementation The decision tree classifier was implemented from scratch to provide a deeper understanding of the process. The implementation follows these key steps:

1. **Gini Impurity Calculation:** The Gini impurity measures the impurity of a dataset based on the distribution of class labels. It is computed as:

$$Gini = 1 - \sum (\text{probabilities of each class})^2$$

This function (`gini_impurity`) is used to evaluate the quality of splits during tree construction.

2. **Finding the Best Split:** The `find_best_split` function identifies the feature and threshold that best split the dataset by minimizing the weighted average of Gini impurities across the subsets. The algorithm:
 - Iterates through each feature and its unique thresholds.
 - Splits the dataset into two groups: left (values $\leq$ threshold) and right (values $>$ threshold).
 - Computes the Gini impurity for the left and right groups and calculates the weighted average impurity.

- Updates the best split if a lower impurity is found.
3. **Building the Tree:** The `build_tree` function recursively builds the decision tree:
 - If all labels in the current subset are the same, a leaf node is created with that class.
 - If a maximum depth is reached or no further splits can be made, a leaf node is created with the majority class.
 - Otherwise, the function finds the best split and recursively builds left and right child nodes.
 4. **Prediction:** The `predict` function traverses the decision tree to classify each data point:
 - `predict_one(tree, x)` processes a single data point by recursively traversing the tree based on feature values and thresholds. When it reaches a leaf node, it returns the class stored there.
 - `predict(tree, X)` applies `predict_one` to every data point in the dataset `X`, returning an array of predicted classes for all data points.
 5. **Evaluation:** The custom-built tree was trained and tested on the Fashion dataset achieving 76.80%

4.2 M2 - Feed-Forward Neural Networks

4.2.1 Introduction

Feed-forward neural networks (FFNN) are artificial neural networks where data flows from input nodes, through hidden layers, to output nodes. In this project, we address multiclass classification using an FFNN with an input layer (112 dimensions via PCA), multiple hidden layers using ReLU activation, and an output layer with softmax activation to predict probabilities for the five clothing categories. Training uses forward and backward propagation to compute predictions, calculate loss, and update parameters.

4.2.2 Data Preparation

Principal Component Analysis Principal Component Analysis (PCA) reduced data dimensionality to 112 components, balancing variance explained and model performance..

Feature Scaling Features were standardized using the `StandardScaler` from `scikit-learn` to ensure numerical stability. Standardization involves scaling the pixel intensity values of the grayscale images to have a mean of 0 and a standard deviation of 1.

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Label Encoding Labels were one-hot encoded for classification. In this method, each class label is transformed into a binary vector where the index corresponding to the class is marked as 1, and all others are set to 0. This ensures compatibility with the categorical cross-entropy loss function and allows the network to output probabilities for each class.

4.2.3 Methods

We tried to ensure consistency between both implementations, making the two networks as identical as possible. To keep the project report concise, we will focus on explaining the details of the FFNN and highlight all the essential information in this section.

Weight and Bias Initialization Weights were initialized using He-initialization to ensure stable variance and prevent exploding or vanishing gradients. This involves drawing weights from a normal distribution with a mean of 0 and variance of $\frac{2}{n_{\text{inputs}}}$ where n_{inputs} is the number of neurons in the input layer to the current layer.

Biases were initialized to 0, which is a standard choice.

Network Structure Our network has 6 layers in total:

- **Input Layer:** It takes inputs with the specified number of dimensions, so 112 in our case because we used PCA.
- **Hidden Layers:** 4 hidden layers with 128, 64, 32 and 16 neurons in each layer, respectively.
 - **Activation Function:** The hidden layers use *ReLU* as the activation function to introduce non-linearity and allow the model to learn complex patterns.
- **Output Layer:** Produces the final output of the network and has 5 neurons, one for each clothing type.
 - **Activation Function:** The output layer uses the *Softmax* as the activation function, which outputs a probability distribution over the 5 clothing categories.

Forward Propagation Forward propagation is the process of passing the input data through the network to compute the output (predictions) and calculating the loss/cost. It computes the activations for each layer iteratively, starting from the input layer. In the hidden layers, it calculates the weighted sums ($Z = A \cdot W + b$) and applies the ReLU activation function to introduce non-linearity. In the output layer, the final predictions are computed by applying the softmax function to the last layer's activations, giving probabilities for each class.

Backpropagation Backward propagation is the process of computing gradients of the loss function with respect to the model's parameters and updating the parameters to minimize the loss by propagating backward through the network. The calculated gradients are then used to update the weights and biases using gradient descent.

Optimization Stochastic Gradient Descent (SGD) is used as an optimization algorithm to train the network by iteratively updating the parameters (weight and biases). It estimates gradients based on small random subsets of data called mini-batches. We chose 32 as the batch size as that provides a good balance between computational efficiency and accuracy.

Regularization Dropout and early stopping help prevent overfitting. Dropout reduces reliance on specific neurons by deactivating them during training. Dropout was applied selectively, with rates of 0.2 on outer layers.

Early stopping monitors the model's performance on a validation set during training. If the validation loss does not improve for a specified number of epochs (patience), training is stopped to prevent overfitting. We used patience of 5.

4.2.4 M2 implementation using library

To implement the FFNN using a pre-existing library, we used the `Sequential` class from `Tensorflow` to define the FFNN structure such as the layers with their activation functions and sizes. Dropout was implemented using `Dropout*` class and added after specific layers to prevent overfitting. To optimize parameters during training, we used Stochastic Gradient Descent `SGD*` with a learning rate of 0.01, as that is a good default. The loss function used was the "Categorical Cross-Entropy". `Early Stopping*` was also implemented to stop training when validation performance no longer improved. The weights were initialized using `He-initialization*` and the biases were initialized to zero as that is the default for `Dense` layers.

* All from the `TensorFlow` library.

```
initializer = HeNormal(seed=random_seed)
early_stopping = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)
model = Sequential([
    Dense(128, activation='relu', input_dim=n_components, kernel_initializer=
        initializer),
    Dropout(0.2, seed=random_seed), Dense(64, activation='relu', kernel_initializer=
        initializer),
    Dense(32, activation='relu', kernel_initializer=initializer),
    Dense(16, activation='relu', kernel_initializer=initializer),
    Dropout(0.2, seed=random_seed), Dense(5, activation='softmax', kernel_initializer=
        initializer)])
model.compile(optimizer=SGD(learning_rate=0.01), loss='categorical_crossentropy',
    metrics=['accuracy'])
```

The model was trained on the PCA-transformed and scaled data using `TensorFlow`'s `fit` method. The number of epochs was set to 100 to allow sufficient training time while enabling early stopping to determine the optimal stopping point. An 80/20 validation split was used, with 80% of the data used for training and 20% reserved for validation.

```
model.fit(X_pca, y_train_categorical, epochs=100, batch_size=32, validation_split=0.2,
    callbacks=[early_stopping])
```

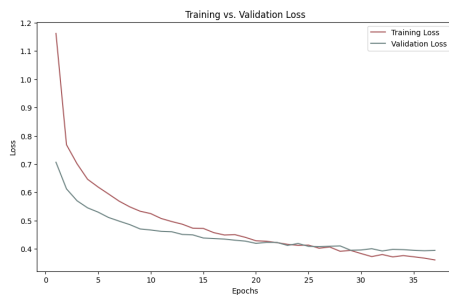


Figure 7: Training vs. Validation Loss

Test Accuracy: 0.84

Classification Report:

	precision	recall	f1-score	support
T-shirt/top	0.83	0.78	0.81	1000
Trouser	0.98	0.96	0.97	1000
Pullover	0.82	0.89	0.85	1000
Dress	0.86	0.93	0.89	1000
Shirt	0.72	0.66	0.69	1000
accuracy			0.84	5000
macro avg	0.84	0.84	0.84	5000
weighted avg	0.84	0.84	0.84	5000

Figure 8: Evaluation of the model on the test data

Model Evaluation The training and validation loss (Figure 7) show smooth convergence over the training epochs, indicating stable learning without significant overfitting. Both losses decrease steadily, with validation loss closely following training loss, which suggests that the model generalizes well to unseen data.

The overall test accuracy is 84%, which is reasonable performance.

The detailed classification report shows good performance for most categories, but comparatively lower performance for the "shirt" category. This may indicate that shirts share features with

other categories, making them harder to classify. This was mentioned at the beginning of this project report and shown in Figure 3.

4.2.5 M2 implementation from scratch

The model was implemented as a class with the following attributes:

```
class FFNN:
    def __init__(self, input_dim, layer_sizes, output_dim, learning_rate
                 =0.01, epochs=10, batch_size=32, early_stopping=False, patience=5,
                 dropout_rates=None, random_seed=42):
```

The class has several methods, most notably **forward_propagation**, which computes the activations for each layer, applying ReLU activation in the hidden layers and softmax in the output layer, **backward_propagation** which calculates gradients of the loss function with respect to weights and biases, propagating the error backward and adjusting for dropout. and **train** which is responsible the training process, dividing data into batches, updating weights using gradient descent, and monitoring performance with early stopping.

Weights are initialized using He-initialization and biases are initialized to zero. Dropout is implemented with customizable rates for each hidden layer. Early stopping is integrated with a patience parameter, saving the best-performing weights and biases to avoid overfitting.

The model was defined and trained with the PCA transformed and scaled data, using the same parameters as the model implemented with **TensorFlow** library. Just as the first implementation, the number of epochs was set to 100. An 80/20 validation split was used, which is set in the **train** method of the **FFNN** class.

```
model = FFNN(input_dim=n_components, layer_sizes=[128, 64, 32, 16], output_dim=5,
             learning_rate=0.01, epochs=100, batch_size=32, early_stopping=True, patience=5,
             dropout_rates=[0.2, 0.0, 0.0, 0.2], random_seed=1616)

model.train(X_train_pca, y_train_encoded)
```



Figure 9: Training vs. Validation Loss

Test Accuracy: 0.80

Classification Report:

Class	Precision	Recall	F1-Score	Support
T-shirt/top	0.78	0.75	0.76	1000
Trouser	0.98	0.95	0.96	1000
Pullover	0.83	0.76	0.79	1000
Dress	0.87	0.87	0.87	1000
Shirt	0.59	0.69	0.64	1000
Accuracy	0.80			
Macro Avg	0.81	0.80	0.81	
Weighted Avg	0.81	0.80	0.81	

Figure 10: Evaluation of the model on the test data

Model Evaluation The training and validation loss (Figure 9) show a similar smooth convergence over the training epochs as the library implementation (Figure 7). Both losses decrease steadily, and the validation loss closely tracks the training loss, indicating stable learning and reasonable generalization to unseen data. However, the gap between training and validation loss is slightly larger in this scratch implementation, which could suggest a bit more overfitting compared to the library based model.

The overall test accuracy for this implementation is 80%, slightly lower than the 84% achieved by the library implementation, but still very reasonable considering this model was developed from scratch.

The scratch implementation shows results that closely match those of the library based model, validating its correctness.

4.3 M3 - Support Vector Machines

4.3.1 Introduction

For our third model we have decided to work with a support vector machine method.

Support Vector Machine is a learning algorithm for classification and regression. It works by finding a hyperplane that best separates data into classes while maximizing the margin between them. For non-linear data, SVM uses kernel functions to map it into a higher-dimensional space for better separability.

4.3.2 Hyper-parameter tuning

C is a regularization parameter that controls the trade-off between maximizing the margin and minimizing classification errors. Fine-tuning C prevents both overfitting and underfitting.

Gamma defines the reach of each training point in influencing the decision boundary and optimizing this parameter ensures better performance on unseen data.

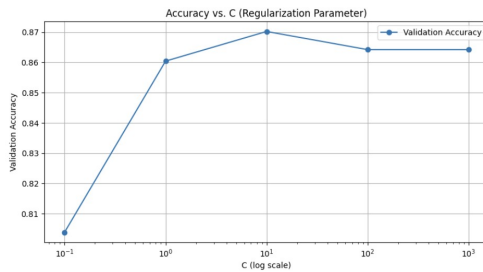


Figure 11: C values against Accuracy

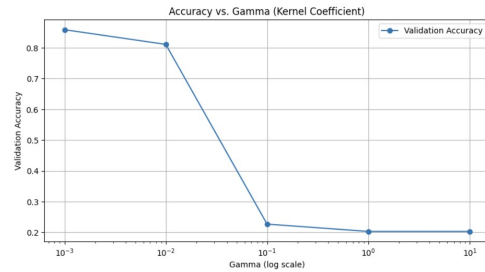


Figure 12: Gamma values against Accuracy

As seen in Figure 11 the values of C that attain the highest accuracy lie in-between 10^0 and 10^2 , similar for Gamma in Figure 12, the best values fall between 10^{-3} and 10^{-2} . These values were used for *RandomizedSearchCV* function which further fits different values in given range to find the single best value. The model was then adjusted with both of these optimized values.

Best Parameters: {'kernel': 'rbf', 'gamma': 0.0014677992676220691, 'C': 2.51188643150958}

Figure 13: The values determined by *RandomizedSearchCV* function

4.3.3 Implementation

We have decided to work with scikit-learn library for its well utilized SVM model. A baseline SVM as well as a fine-tuned counterpart were trained, tested and the results were compared for further evaluation.

4.3.4 Model evaluation

Baseline Accuracy: 0.8482

	precision	recall	f1-score	support
0	0.80	0.83	0.81	1000
1	1.00	0.95	0.97	1000
2	0.84	0.87	0.85	1000
3	0.87	0.92	0.89	1000
4	0.73	0.67	0.70	1000
accuracy			0.85	5000
macro avg	0.85	0.85	0.85	5000
weighted avg	0.85	0.85	0.85	5000

Figure 14: Baseline SVM classification

Tuned Model Accuracy: 0.8548

	precision	recall	f1-score	support
0	0.82	0.83	0.82	1000
1	0.99	0.96	0.98	1000
2	0.86	0.86	0.86	1000
3	0.88	0.92	0.90	1000
4	0.73	0.71	0.72	1000
accuracy			0.85	5000
macro avg	0.86	0.85	0.85	5000
weighted avg	0.86	0.85	0.85	5000

Figure 15: Fine-tuned SVM classification

The fine-tuned SVM model achieved an accuracy of 85.48%, which is a slight improvement over the baseline SVM accuracy of 84.82%. The tuned model performed well across most classes, with particularly high precision, recall, and F1-scores for class 1 and class 3. However, the performance for class 4 was lower, with a precision of 0.73 and a recall of 0.71, suggesting some difficulty distinguishing this class from others.

5 Interpretation and Discussion of Results

Model	Accuracy (%)
Decision Trees (scratch)	76.80
Decision Trees (library)	80.02
Feed-Forward Neural Networks (scratch)	80.00
Feed-Forward Neural Networks (library)	84.00
Support Vector Machines (library)	85.48

Table 5: Comparison of model accuracies

5.1 Model Performance Comparison

Decision Trees: The scratch implementation achieved 76.80% accuracy, while the library implementation improved to 80.02% due to additional hyperparameters like `min_leaf` and `min_split`, which prevent overfitting and were not included in the scratch version.

Feed-Forward Neural Networks: The scratch implementation reached 80.00%, while the library version improved to 84.00%, benefiting from optimized training algorithms and regularization.

Support Vector Machines (SVM): SVM achieved the highest accuracy of 85.48%, demonstrating its effectiveness for high-dimensional data, though it may face scalability challenges with larger datasets.

5.2 Conclusion

Support Vector Machines had the best accuracy, proving their strength in handling high-dimensional data. Feed-Forward Neural Networks also performed well, with library implementations benefiting from optimizations. Decision Trees, while less accurate, provided a simpler baseline. Exploring advanced models like Convolutional Neural Networks could further enhance results.